

NLP Pipeline Architecture: Deep Technical Analysis

Documentation

February 5, 2026

Overview

This document outlines the architecture of the NLP MENA project, detailing the role, logic, and technical implementation of each core file in the pipeline order.

1 File Roles & Technical Analysis

1. `src/config.py`

Role: Centralized Configuration Management.

Logic:

- Acts as the single source of truth for all project parameters (Database credentials, Model paths, Batch sizes).
- Dynamically loads environment variables from a `.env` file using `python-dotenv`.
- Constructs complex strings (like the Database Connection URI) automatically to prevent formatting errors elsewhere.

Technicality:

- Uses the `Pathlib` library to resolve absolute paths relative to the project root, ensuring the code runs on any machine without path issues.
- Implements a robust `_getenv` helper to handle data type casting and default values safely.
- Exposes configuration as static class attributes (e.g., `Config.DB_URI`) for easy import.

2. `src/db_config.py`

Role: Database Abstraction Layer (DAO) & Infrastructure.

Logic:

- Manages the lifecycle of the MySQL connection, ensuring connections are opened, reused (pooled), and closed properly.
- **Schema Management:** Contains the DDL (Data Definition Language) for the NLP result tables and creates them if they do not exist.
- Provides specialized methods for writing results (e.g., `save_language_detection`) to decouple SQL logic from the main pipeline.

Technicality:

- Built on top of **SQLAlchemy Core**, utilizing `create_engine` with `QueuePool` for efficient connection pooling.
- Implements `pool_pre_ping=True` to automatically handle and reset stale database connections (e.g., "MySQL Server has gone away").
- Uses parameterized SQL queries (binding variables like `:id`, `:lang`) to prevent SQL injection.

3. src/preprocessing.py

Role: Text Normalization Engine (Arabic Specialized).

Logic:

- Transforms raw, noisy text into clean input suitable for machine learning models.
- Implements a distinct "layered" approach: `preprocess_for_lang_detect` preserves sentence structure for context, while `preprocess_for_keywords` aggressively strips noise.

Technicality:

- **Unicode Normalization:** Uses `unicodedata.normalize('NFC', text)` to compose decomposed characters.
- **Code Point Filtering:** Explicitly defines Regex ranges for Arabic Unicode blocks:
 - Basic Arabic: `[\u0600-\u06FF]`
 - Arabic Supplement: `[\u0750-\u077F]`
- **Normalization Rules:** Maps multiple variants of Alef (, ,) to a bare Alef () and Yaa () to () to reduce vocabulary dimensionality for models.
- **Heuristics:** Includes an `arabic_ratio` algorithm that calculates the density of Arabic characters versus total characters to filter out non-Arabic noise.

4. src/language_detection.py

Role: Model Wrapper / Inference Interface.

Logic:

- Encapsulates the **FastText** model, ensuring the heavy model file is loaded into memory only once per process.
- Provides a standardized return object handling both the raw model output and the clean ISO code.

Technicality:

- **Return Types:** Uses a frozen `@dataclass` for the `LanguageDetection` result. This ensures immutability and memory efficiency compared to standard dictionaries.
- **Input Validation:** Includes robust type checking to handle `None` or non-string inputs gracefully, returning a default "unk" (Unknown) state instead of crashing.
- **Dependency Management:** Wraps the C++ based `fasttext` bindings. It employs path resolution logic using `pathlib.Path(__file__)` to locate the binary `.ftz` model file relative to the script location, making the module portable.

5. main.py

Role: ETL Pipeline Orchestrator.

Logic:

- **Extract:** Fetches data from MySQL. Instead of iterating row-by-row (slow), it fetches batches into a Pandas DataFrame.
- **Transform:** Vectorized application of the preprocessor across the entire DataFrame column before passing to the model loop.
- **Load:** Writes results back to the database and dumps CSV snapshots for auditability.

Technicality:

- **Pandas-SQL Bridge:** Uses `pd.read_sql` with the SQLAlchemy engine object, allowing for efficient memory management of the result set.

- **Batch Processing:** fetches raw strings, applies `df.apply(preproc.func)` for CPU-efficient string manipulation, and then iterates tuples for the inference step.
- **Artifact Management:** dynamically creates a timestamped directory (e.g., `experiments/2026-02-05_smk`) to store logs and CSVs, ensuring every run is reproducible and isolated.